

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Обработка сетевых пакетов

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Задание.....	4
3 Краткое описание хода работы	5
4 Исходный код.....	6
5 Реализация дополнительных заданий.....	12
6 Результат работы с примерами	13
7 Выводы	14

1 Цель работы

Цель данной лабораторной работы заключается в реализации обработчика сетевых пакетов, что позволит глубже понять механизмы обработки данных в условиях ограниченных ресурсов. В рамках работы будет проведён детальный анализ работы системы, учитывающей размеры буфера и временные характеристики обработки пакетов. Особое внимание будет уделено разработке симулятора, который моделирует порядок обработки пакетов с возможными дополнительными усложнениями, такими как приоритеты.

2 Задание

Для выполнения лабораторной работы необходимо разработать программу, реализующую симуляцию обработки сетевых пакетов в соответствии с заданными параметрами. Программа должна принимать на вход размер буфера и число пакетов через консоль. Каждая из последующих строк должна содержать время поступления пакета и время его обработки.

После считывания входных данных программа должна моделировать процесс обработки пакетов, учитывая, что компьютер обрабатывает пакеты в порядке их поступления, и при этом не может превышать размер буфера. Если пакет поступает, когда буфер уже заполнен, он должен быть отброшен.

Также необходимо выбрать два дополнительных условия. В моем случае выбран: Приоритет пакетов, Статистика и анализ.

Суть первого задания ввести систему приоритетов для пакетов. Пакеты с более высоким приоритетом должны обрабатываться первыми, даже если они поступили позже других пакетов. При равном приоритете пакеты обрабатываются в порядке поступления.

Суть второго задания добавить сбор статистики по времени ожидания пакетов, проценту отброшенных пакетов и времени простоя процессора.

3 Краткое описание хода работы

В ходе выполнения лабораторной работы была разработана программа для симуляции обработки сетевых пакетов с учетом их приоритетов и времени обработки.

Ввод данных: Программа начинает с запроса у пользователя размера буфера и количества пакетов. Для каждого пакета пользователь вводит время поступления, время обработки и приоритет. Ввод данных осуществляется через консоль, и программа проверяет корректность введенных значений.

Сравнение и сортировка пакетов: Все введенные пакеты собираются в массив объектов, где каждый объект содержит параметры пакета (время поступления, время обработки и приоритет). Затем массив пакетов сортируется: сначала по приоритету (в порядке убывания), а при равных приоритетах — по времени поступления.

Обработка пакетов: Программа проходит по отсортированному массиву пакетов и симулирует их обработку. Если пакет поступает, когда буфер заполнен, он считается отброшенным. В процессе обработки программа фиксирует время ожидания для каждого пакета, а также общее время работы системы.

Расчет статистики: После обработки всех пакетов программа вычисляет и выводит статистические данные, включая общее количество пакетов, количество обработанных и отброшенных пакетов, среднее время ожидания и время простоя процессора.

Вывод результатов: Результаты обработки, включая статус каждого пакета (отброшен или обработан), а также статистику, выводятся в консоль.

Таким образом, программа демонстрирует основные принципы обработки сетевых пакетов, учитывая приоритеты и ограничения по размеру буфера, а также анализирует эффективность алгоритма обработки.

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
// Функция валидации блока size n пакетов.
function isValid(n) {
    return n > 0 && n % 1 === 0;
}

// Функция сравнения двух массивов по size arrive и отсева непоточных
элементов массивов с учетом приоритета.
function compareBuffer(arrayOfArrival, arrayOfDuration, arrayOfPriority) {
    let newArrayOfArrival = [];
    let newArrayOfDuration = [];
    let newArrayOfPriority = [];
    let setRepeated = new Set([]);
    let arrayRepeated = Array.from(setRepeated);

    // Собираем массив пакетов со всеми параметрами
    let packets = [];
    for (let j = 0; j < arrayOfArrival.length; j++) {
        packets.push({
            arrival: arrayOfArrival[j],
            duration: arrayOfDuration[j],
            priority: arrayOfPriority[j]
        });
    }

    // Сортировка пакетов по приоритету (более высокий приоритет - पहले), а
    при равных приоритетах по времени прибытия
    packets.sort((a, b) => {
```

```

    if (a.priority !== b.priority) {
        return b.priority - a.priority; // Чем выше приоритет, тем раньше
        обрабатывается
    } else {
        return a.arrival - b.arrival; // При равном приоритете по времени
        поступления
    }
});

// Отсеивание непоточных элементов и формирование новых массивов
for (let j = 0; j < packets.length; j++) {
    if (packets[j].arrival <= size) {
        if (!arrayRepeated.includes(packets[j].arrival)) {
            newArrOfArrival.push(packets[j].arrival);
            newArrOfDuration.push(packets[j].duration);
            newArrOfPriority.push(packets[j].priority);
            arrayRepeated.push(packets[j].arrival);
        } else {
            newArrOfArrival.push(packets[j].arrival);
            newArrOfDuration.push(-1);
            newArrOfPriority.push(packets[j].priority);
        }
    } else {
        packets[j].duration = -1;
        if (!arrayRepeated.includes(packets[j].arrival)) {
            newArrOfArrival.push(packets[j].arrival);
            newArrOfDuration.push(packets[j].duration);
            newArrOfPriority.push(packets[j].priority);
            arrayRepeated.push(packets[j].arrival);
        } else {

```

```

        newArrOfArrival.push(packets[j].arrival);
        newArrOfDuration.push(-1);
        newArrOfPriority.push(packets[j].priority);
    }
}

}

return [newArrayOfArrival, newArrOfDuration, newArrOfPriority];
}

// Функция ввода данных из консоли.
function getInputData() {
    const readline = require('readline-sync');
    let size = Number(readline.question("Введите размер буфера: "));
    let n = Number(readline.question("Введите число пакетов в буфере: "));
    if (!isValid(size) || !isValid(n)) {
        console.log("Размер введенного пакета или буфера слишком мал");
        process.exit();
    } else {
        let arrayOfArrival = [];
        let arrayOfDuration = [];
        let arrayOfPriority = [];
        for (let i = 0; i < n; i++) {
            let arrival = Number(readline.question("Введите время поступления
пакета: "));
            let duration = Number(readline.question("Введите время обработки
пакета: "));
            let priority = Number(readline.question("Введите приоритет пакета (чем
выше число, тем выше приоритет): "));

```



```

        arrayOfArrival.push(arrival);
        arrayOfDuration.push(duration);
        arrayOfPriority.push(priority);
    }
    return [n, size, arrayOfArrival, arrayOfDuration, arrayOfPriority];
}
};

```

// Функция вывода результата с учётом приоритета и расчёта статистики.

```

function resultHandler(newArrayOfArrival, newArrayOfDuration,
newArrayOfPriority) {

```

```

    let time = -1;
    let totalWaitingTime = 0;
    let processedPackets = 0;
    let totalProcessingTime = 0;
    let idleTime = 0;
    let discardedPackets = 0;

```

```

    for (let i = 0; i < newArrayOfArrival.length; i++) {
        if (newArrayOfDuration[i] === -1) {
            // Пакет отброшен
            discardedPackets++;
            console.log "[" + (i+1) + "] Пакет с приоритетом " +
newArrayOfPriority[i] + " отброшен.");
            continue;
        }

```

```

        if (time < newArrayOfArrival[i]) {
            // Если процессор простаивает
            idleTime += newArrayOfArrival[i] - time;

```

```

    time = newArrayOfArrival[i];
}

// Время ожидания пакета
let waitingTime = time - newArrayOfArrival[i];
totalWaitingTime += waitingTime;

// Время завершения обработки пакета
time += newArrayOfDuration[i];
totalProcessingTime += newArrayOfDuration[i];

processedPackets++;

console.log "[" + (i+1) + "] Операция с приоритетом " +
newArrayOfPriority[i] + " завершилась в " + time + " секунду. Время
ожидания: " + waitingTime + " сек.");
}

// Статистика
const totalPackets = newArrayOfArrival.length;
const totalTime = newArrayOfArrival[newArrayOfArrival.length - 1] +
newArrayOfDuration[newArrayOfDuration.length - 1]; // Общее время работы
системы

const discardedPercentage = (discardedPackets / totalPackets) * 100; // Процент
отброшенных пакетов

const averageWaitingTime = totalWaitingTime / processedPackets; // Среднее
время ожидания пакета

console.log("\n--- Статистика ---");
console.log("Общее количество пакетов: " + totalPackets);

```

```
    console.log("Обработано пакетов: " + processedPackets);
    console.log("Отброшено пакетов: " + discardedPackets + " (" +
discardedPercentage.toFixed(2) + "%)");
    console.log("Среднее время ожидания пакетов: " +
averageWaitingTime.toFixed(2) + " сек.");
    console.log("Время простоя процессора: " + idleTime + " сек.");
}
```

```
let [n, size, arrayOfArrival, arrayOfDuration, arrayOfPriority] = getInputData();
console.log("\nБуфер до обработки:");
console.log(arrayOfArrival);
console.log(arrayOfDuration);
console.log(arrayOfPriority);
console.log("\nБуфер после обработки:");
let [newArrayOfArrival, newArrayOfDuration, newArrayOfPriority] =
compareBuffer(arrayOfArrival, arrayOfDuration, arrayOfPriority);
console.log(newArrayOfArrival);
console.log(newArrayOfDuration);
console.log(newArrayOfPriority);
resultHandler(newArrayOfArrival, newArrayOfDuration, newArrayOfPriority);
```

5 Реализация дополнительных заданий

В данной лабораторной работе было выполнено два дополнительных задания. Проведена реализация приоритетов пакетов и анализ алгоритма сортировки пакетов

Реализация пакетов приоритета добавлением на вход еще одного пункта – приоритет пакета. В алгоритме его главная задача фильтровать порядок пар arrival + duration после ввода и первой фильтрации. Это позволяет запускать определенные пакеты вне очереди, если они проходят по всем критериям.

```
[1] Операция с приоритетом 4 завершилась в 3 секунду. Время ожидания: 0 сек.  
[2] Пакет с приоритетом 3 отброшен.  
[3] Операция с приоритетом 2 завершилась в 4 секунду. Время ожидания: 3 сек.
```

Рисунок 1 – Пример работы приоритета пакета в алгоритме.

Реализация анализа данных состоит из: анализа количества пакетов, обработанных пакетов, отброшенных после фильтрации, среднее время ожидания пакета (сек), время простоя процессора. Статистика позволяет подробно увидеть трансфер пакетов в система.

```
--- Статистика ---  
Общее количество пакетов: 4  
Обработано пакетов: 2  
Отброшено пакетов: 2 (50.00%)  
Среднее время ожидания пакетов: 1.50 сек.  
Время простоя процессора: 2 сек.
```

Рисунок 2 – Пример статистики пакетов в алгоритме.

6 Результат работы с примерами

Ввод значений пакетов в алгоритм производится через консоль пользователем, задается size, arrival, duration, priority.

```
Введите размер буфера: 3
Введите число пакетов в буфере: 4
Введите время поступления пакета: 5
Введите время обработки пакета: 4
Введите приоритет пакета (чем выше число, тем выше приоритет): 2
Введите время поступления пакета: 0
Введите время обработки пакета: 8
Введите приоритет пакета (чем выше число, тем выше приоритет): 1
Введите время поступления пакета: 5
Введите время обработки пакета: 3
Введите приоритет пакета (чем выше число, тем выше приоритет): 2
Введите время поступления пакета: 1
Введите время обработки пакета: 2
Введите приоритет пакета (чем выше число, тем выше приоритет): 3

Буфер до обработки:
[ 5, 0, 5, 1 ]
[ 4, 8, 3, 2 ]
[ 2, 1, 2, 3 ]
```

Рисунок 1 – Ввод

Вывод значений в консоль после обработчика и вывод дополнительных заданий (статистики и приоритетов).

```
Буфер после обработки:
[ 1, 5, 5, 0 ]
[ 2, -1, -1, 8 ]
[ 3, 2, 2, 1 ]
[1] Операция с приоритетом 3 завершилась в 3 секунду. Время ожидания: 0 сек.
[2] Пакет с приоритетом 2 отброшен.
[3] Пакет с приоритетом 2 отброшен.
[4] Операция с приоритетом 1 завершилась в 11 секунду. Время ожидания: 3 сек.

— Статистика —
Общее количество пакетов: 4
Обработано пакетов: 2
Отброшено пакетов: 2 (50.00%)
Среднее время ожидания пакетов: 1.50 сек.
Время простоя процессора: 2 сек.
```

Рисунок 2 – Вывод

7 Выводы

В результате выполнения лабораторной работы была успешно разработана и реализована программа для симуляции обработки сетевых пакетов с учетом их приоритетов. Программа позволяет эффективно управлять ресурсами компьютера, обрабатывая пакеты в порядке их поступления и учитывая размеры буфера. С помощью данной симуляции удалось продемонстрировать, как различные параметры, такие как время поступления и приоритет пакетов, влияют на общую производительность системы.

Анализ результатов работы программы показал, что использование системы приоритетов значительно повышает эффективность обработки пакетов. Среднее время ожидания пакетов и процент отброшенных пакетов служат важными индикаторами качества работы алгоритма. Полученные статистические данные помогают оценить влияние объема входных данных на время обработки и выявить узкие места в системе. Эти выводы могут быть полезны для дальнейшего изучения алгоритмов обработки данных и оптимизации работы сетевых систем в целом.

В дальнейшем, можно рассмотреть возможность реализации дополнительных функций, таких как многопоточная обработка или динамическое изменение размера буфера, что позволит еще более точно имитировать работу реальных сетевых систем. Такие доработки сделают программу более универсальной и применимой в различных сценариях, что будет полезно для студентов и специалистов в области компьютерных наук.